GRIT — Track Badge Interview Scheduling

PAGE 1 · JOURNEY & PRODUCT DECISIONS

Problem. A skill-ready student needs a clear next step; the interview team needs a reservation system and an actionable eligible/unbooked queue. This prototype covers the Track Badge interview, separate from GRIT final selection. The primary journey is AI Product Mastery — Novice; AI Systems Mastery — Novice and Product Specialist demonstrate multiple journeys and sequential prerequisites. No Miles are invented.

Four independent dimensions. Eligibility = cleared requirements + valid previous badge + no integrity restriction. Reservation = booked/cancelled/no-show. Attempt = completed + nullable PASS/FAIL/RETAKE_LATER. Award = valid/invalidated. Dashboard status is derived; historical PASS never becomes FAIL when an award is revoked.

Current state	Trigger / actor	Next state	Key rule / effect
Locked	Skills complete / progress service	Eligible, unbooked	Every skill and prerequisite must be valid
Eligible / cancelled / retake	Book / student	Booked	Future slot; no student/interviewer overlaps
Booked	Reschedule / student	Booked, new attempt	Reserve new + release old atomically; old attempt cancelled
Booked	Cancel / student or team	Cancelled	Student before start; history retained; slot released
Booked	Complete / assigned interviewer or team	Awaiting result	After end; explicit audited demo simulation available
Awaiting result	PASS / assigned interviewer or team	Passed + valid award	Recheck eligibility; award the matching badge
Awaiting result	FAIL / assigned interviewer or team	Failed	No award; retry needs team authorization
Awaiting result	RETAKE_LATER / assigned interviewer or team	Retake required	Next booking creates a new numbered attempt
Booked	No-show / team	Missed	After end; explicit demo simulation; retry needs approval
Failed / missed	Authorize retry / team	Retake required	Audited one-attempt authorization
Valid award	Confirm malpractice / team	Invalidated award	Require reason; preserve PASS; block future prerequisites
Invalidated award	Overturn finding / team	Valid award	Require reason; append audit event

Handled edge cases. Guards reject stale/occupied/past slots, duplicate bookings, overlapping student or interviewer reservations, unauthorized results and repeated result submission. Failed rescheduling preserves the original. Interview history protects availability from edit/delete, even after cancellation. Eligibility loss after booking preserves the reservation for team review and blocks result entry. Invalidated prerequisite awards block future progression; existing downstream awards need manual review. Refresh restores browser data. Scenario loading requires confirmation and replaces data; persona switching preserves it.

Demo assumptions, not NIAT policy. 30 minutes; UTC storage/IST display; students cancel or reschedule before start; RETAKE_LATER grants a fresh attempt; FAIL/no-show need team approval. The explicit “Demo: complete interview” bypasses time only and is audited. Internal notes stay out of student views. Saved results cannot be overwritten; wrong-result corrections are escalated. Award restoration is supported and audited.

Confirm with NIAT. Cancellation notice/penalties, retry limits/cooldowns, interviewer assignment rules, student feedback, pending-result escalation, appeal/correction authority and downstream award treatment. Skill progress is seeded, not connected to real contests.

Relational design. All IDs are UUID primary keys; mutable rows carry created_at/updated_at. The demo uses equivalent typed objects and explicit transition functions; SQL is a production design, not a running backend.

Table	Important fields / relationships
students; interviewers	id, name; students.cohort, integrity_restricted
tracks; badges	badges.track_id → tracks; tier; previous_badge_id → badges
skills; badge_skill_requirements	requirement(badge_id, skill_id, required_level), composite PK
student_skill_progress	(student_id, skill_id) PK, cleared_level, verified_at
interview_slots	id, interviewer_id → interviewers, start_at/end_at timestamptz; each row is 30-minute availability
interview_attempts	student_id, badge_id, slot_id FKs; attempt_no; booking_status; outcome; result_at; notes; retry_authorized
badge_awards	student_id, badge_id, attempt_id FKs; validity; awarded_at; invalidated_at/reason
integrity_flags	student_id, award_id FKs; reason; confirmed_by; confirmed_at; overturned_at/by
audit_events	actor_id/role, action, entity_id, reason, created_at; append-only

Cardinalities & constraints. Track 1:N badges; badges N:M skills; student 1:N attempts per badge; interviewer 1:N slots; attempt 0:1 award. UNIQUE(student,badge,attempt_no); UNIQUE(student,badge) award; UNIQUE(attempt_id) award. Partial UNIQUE(student,badge) for active reservations and pending results; partial UNIQUE(slot_id) for booked reservations. Index slots(interviewer,start), attempts(student,badge), attempts(status), flags(student), audit(entity,created_at). Store outcome separately from booking status and award validity.

Production atomicity & access. Book/reschedule in one transaction: lock the student journey and slot, revalidate eligibility, claim reservation, release old reservation, append audit. PostgreSQL exclusion constraints on active student and interviewer timestamptz ranges prevent cross-slot overlaps; slot FK and partial uniqueness prevent slot reuse. The reservation range is a transactionally maintained projection of slot times. Idempotency keys reject duplicate requests; return a conflict with fresh slots. Server-side role/ownership checks, immutable result records, flag decisions and durable audit events are mandatory. Browser guards cannot guarantee cross-client concurrency or security.

Three exact messages displayed in the UI.

1. “You're ready for your Track Badge interview! You've cleared all required skills. Book your interview to take the final step toward earning your badge.” CTA: **Book Interview**.

2. “Your interview is booked for {date} at {time} IST. You can view the details or reschedule from your dashboard.” CTA: **View Interview**. Date/time are formatted at runtime, for example 12 Nov 2026 / 10:00 am.

3. “Congratulations! You've passed your interview and earned the {track} — {badge} Track Badge.” CTA: **View My Badge**. Main journey substitutes AI Product Mastery / Novice.

Before → after. Scenario controls load incomplete, eligible, booked, pending, passed, failed, retake and invalidated states consistently across home, booking, details and badges. Home changes its headline, guidance, CTA, skill progress and badge appearance. Real workflow: Student books → assigned Interviewer sees appointment → explicit demo completion → PASS → Student earns badge → Team confirms malpractice → award invalidated, historical PASS preserved. Team filters eligible/unbooked journeys and sees derived counts.

Prototype: https://grit-interview-hub-niat.chattytick8.chatgpt.site (public, no login). Open `/#demo` for the reviewer guide. React + strict TypeScript + Vite; deterministic relative-date seeds, 12 fictional students and 3 interviewers; browser-local persistence, no login. No cross-user sync, production authentication, notifications or meeting integration. Source research: official GRIT rules, how-it-works, track progression and homepage, reviewed 11 November 2026; dashboard is login-only.